



Golang Gin: SSE dan WebSocket

Bahas *SSE*, *WebSocket*, beda konsep, alur dasar, dan latihan praktik menengah.



Juara Coding

June 2026



Komunikasi Real-Time

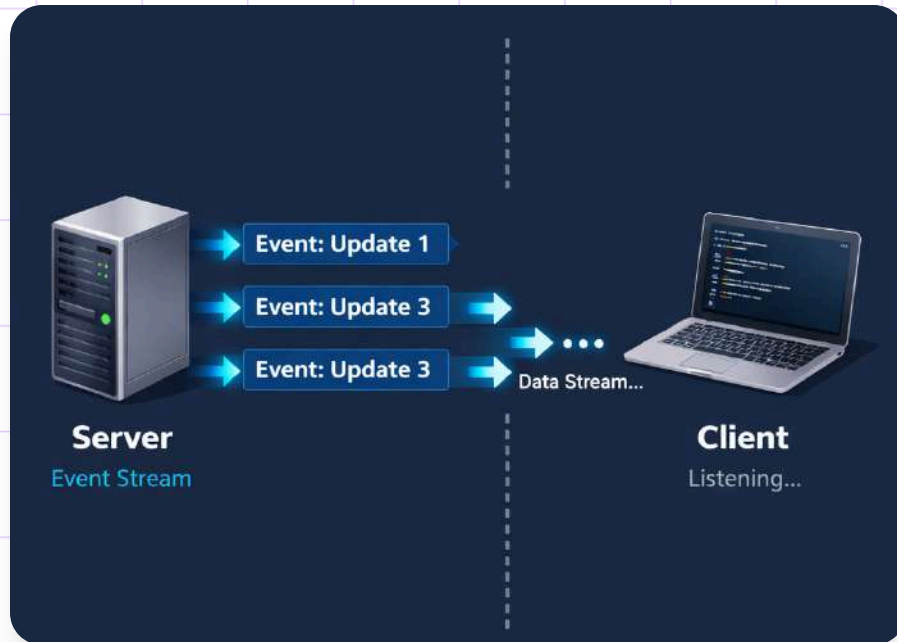
Aplikasi modern memerlukan pembaruan data tanpa refresh, seperti notifikasi, dashboard, chat, dan pelacakan proses dengan SSE atau WebSocket.



Apa Itu Server-Sent Events

Server mengirim update terus-menerus ke client lewat koneksi HTTP terbuka. Cocok untuk notifikasi, status, log, dan streaming data ringan.

Karakteristik SSE

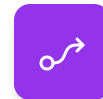


SSE memakai HTTP standar dan *text/event-stream* untuk aliran data satu arah dari server ke client.



HTTP Standar

Menggunakan protokol HTTP biasa dengan content-type *text/event-stream* yang mudah diterapkan.



Satu Arah

Komunikasi berjalan dari server ke client tanpa mekanisme interaksi dua arah.



Mudah untuk Streaming

Cocok untuk streaming teks, event periodik, dan pembaruan data yang sederhana.

Contoh Alur SSE di Gin

```
SSE Stream in Gin
ta: Connected to SSE stream.
1
ta: Message 1 - Hello from Gin!
2
ta: Message 2 - Current Time: 2024-04-24 10:30:00
3
ta: Message 3 - Random Number: 42
4
ta: Message 4 - Stream event update...
5
ta: Message 5 - Goodbye from Gin!
```

01 Koneksi Tetap Terbuka

Client mengakses endpoint SSE, lalu server menjaga koneksi tetap aktif agar event dapat dikirim terus-menerus.

02 Event Dikirim Berkala

Backend mengirim data seperti waktu, status antrian, atau progres pekerjaan secara periodik ke client.

03 Header dan Flush

Handler Gin menyiapkan header respons SSE, lalu melakukan flush setiap kali event baru dikirim.

```
ecting to SSE endpoint...
```

```
ent: message 1
```

```
ta: This is- the first event.
```

```
ent: message 2
```

```
ta: Here is- the second event.
```

```
ent: message 3
```

```
ta: Another event has arrived.
```

```
ent: message 4
```

```
ta: This is the fourth event.
```

```
ent: message 5
```

```
ta: Final event received.
```

```
events processed. Closing connection.
```

Contoh Implementasi SSE

Endpoint Go dengan Gin ini mengirim lima event SSE berurutan ke client melalui koneksi yang tetap terbuka.

Kelebihan dan Keterbatasan SSE



Sederhana dan ringan

SSE mudah diimplementasikan, hemat sumber daya, dan cocok untuk streaming event satu arah.



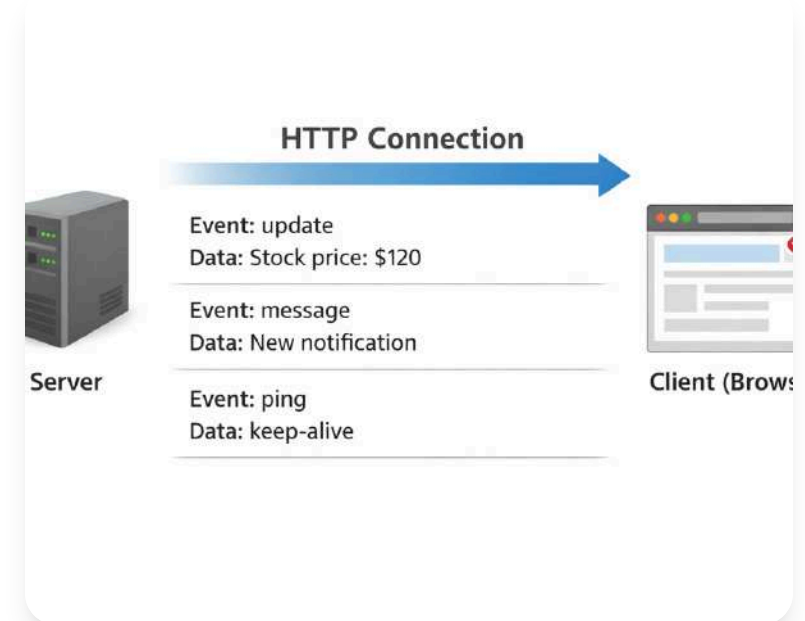
Mudah terintegrasi HTTP

Karena berjalan di atas HTTP, SSE mudah dipakai pada arsitektur web yang sudah ada.



Kurang cocok dua arah

Client tidak dapat mengirim pesan pada koneksi yang sama, sehingga kurang ideal untuk chat interaktif.

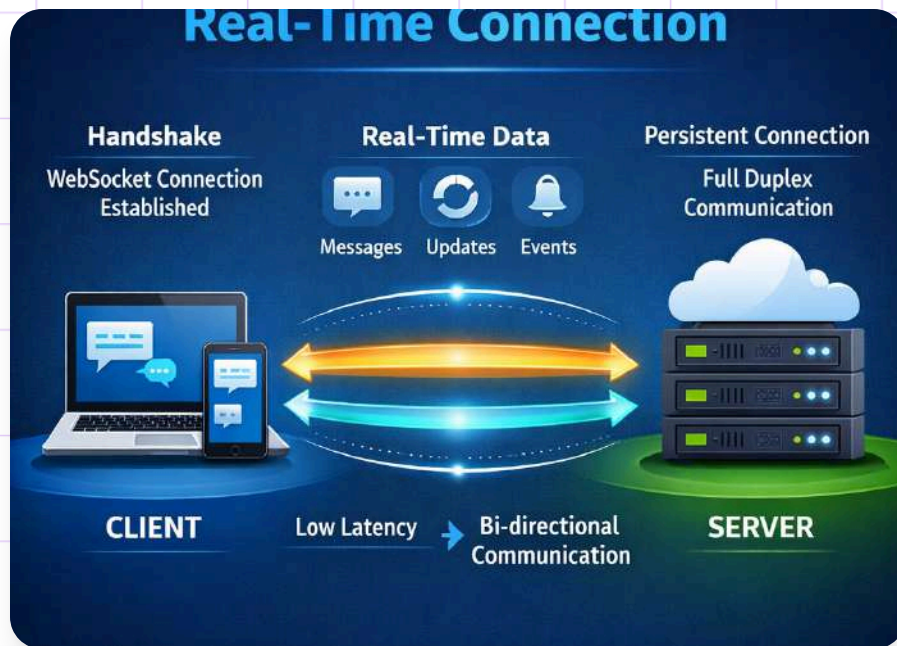




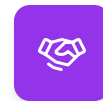
Apa Itu WebSocket

WebSocket adalah protokol untuk komunikasi dua arah penuh antara client dan server lewat satu koneksi persisten secara real-time.

Karakteristik WebSocket

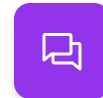


WebSocket dimulai dengan handshake HTTP lalu menjadi koneksi persisten dua arah untuk komunikasi real-time yang lebih fleksibel.



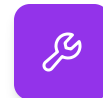
Handshake lalu upgrade

Koneksi diawali lewat HTTP handshake, lalu ditingkatkan menjadi kanal komunikasi persisten.



Komunikasi dua arah

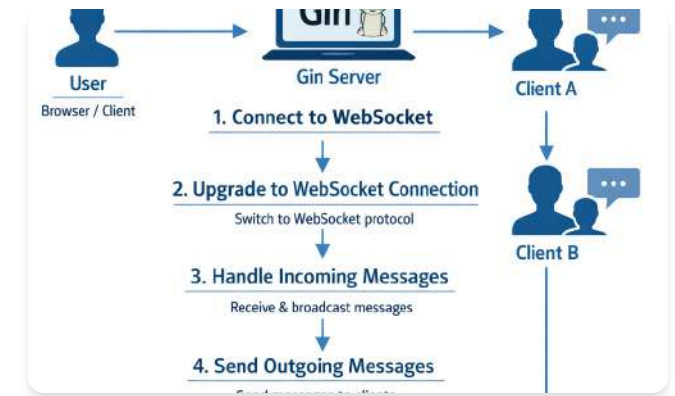
Server dan client dapat mengirim pesan kapan saja setelah koneksi berhasil dibuat.



Fleksibel namun kompleks

Cocok untuk skenario interaktif, tetapi implementasi dan pengelolaan koneksinya lebih rumit.

Contoh Alur WebSocket di Gin



01 Koneksi ke Endpoint

Client terhubung ke endpoint WebSocket, lalu server melakukan upgrade koneksi agar sesi real-time dimulai.

02 Baca dan Balas Pesan

Setelah aktif, backend membaca pesan dari client dan dapat langsung membalas sesuai alur aplikasi.

03 Teruskan ke Client Lain

Pesan juga bisa diteruskan ke client lain. Pola ini umum untuk chat, room diskusi, dan notifikasi dua arah.

```

6  "net/http"
7  "github.com/gorilla/websocket")
8  }
9
10 var upgrader = websocket.Upgrader{
11     CheckOrigin: func(r *http.Request) bool {
12         return true
13     }
14 }
15
16 func echoHandler(w http.ResponseWriter, r *http.Request) {
17     conn, err := upgrader.Upgrade(w, r, nil)
18     if err != nil
19         fmt.Println("Upgrade error", err)
20         return
21     defer conn.Close()
22     for {
23         messageType, msg, err := conn.ReadMessage()
24         if err != nil{
25             fmt.Println("Read error", err)
26         }
27         err =
28             conn.WriteMessage(msgType, msg)
29         if err != nil{
30             fmt.Println("Write error", err)
31         }
32     }
33 }
34
35 func main() {
36     http.HandleFunc("/echo", echoHandler)

```

Contoh Implementasi WebSocket

Kode Go ini membuat endpoint WebSocket sederhana yang menerima pesan lalu mengirim balasan *echo* ke klien secara langsung.

Perbandingan SSE dan WebSocket

Aspek	SSE	WebSocket
Arah komunikasi	Satu arah	Dua arah
Dasar protokol	HTTP	Upgrade dari HTTP
Cocok untuk	Notifikasi, log, status	Chat, game, kolaborasi
Kompleksitas	Lebih sederhana	Lebih kompleks

Pemilihan teknologi bergantung pada kebutuhan interaksi aplikasi.

Kapan Menggunakan SSE



Push Data Satu Arah

Cocok saat server hanya perlu mengirim pembaruan ke client tanpa balasan real-time dari client.



Prioritaskan Kesederhanaan

Pilih SSE ketika implementasi yang sederhana lebih penting daripada komunikasi dua arah yang interaktif.



Monitoring dan Status

Ideal untuk monitoring CPU, status deployment, progres ekspor data, dan notifikasi sistem.



Kapan Menggunakan WebSocket



Komunikasi Dua Arah Aktif

Pilih WebSocket saat client dan backend perlu saling mengirim pesan secara langsung dan berkelanjutan.



Respons Sangat Cepat

Digunakan ketika respons dari client ke backend harus terjadi dalam waktu singkat.

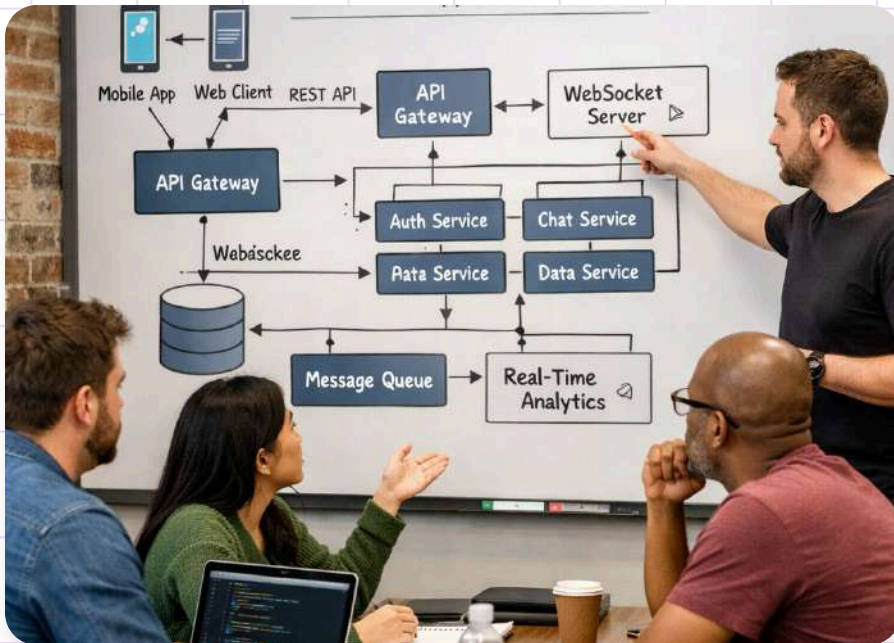


Cocok untuk Real-Time

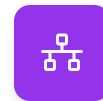
Umum dipakai pada live chat, dashboard kolaboratif, trading, multiplayer game, dan notifikasi interaktif.



Praktik Baik Implementasi



Kelola koneksi, error, timeout, dan resource dengan baik. Pisahkan routing, service, dan handler agar kode mudah diuji dan dikembangkan.



Manajemen Koneksi

Pantau status koneksi, timeout, dan putusnya client agar aliran data tetap stabil.



Penanganan Error

Tangani error secara konsisten agar layanan realtime lebih andal dan mudah dipantau.



Struktur Kode Modular

Pisahkan routing, service, dan handler supaya pengujian dan pengembangan lebih mudah.

Latihan Praktik 1

```
}  
  
func main () {  
    r := gin.Default()           // Endpoint to stream status updates  
    r.GET("/status", statusStream) // Send 5 status updates  
    r.Run(":8080")               with a 2-second interval  
}  
  
func statusStream C *gin.Context {  
    c.Stream(funcw io.Writer) bool {  
        for i := 1; i <= 5; i++ {  
            msg := fmt.Sprintf("data: Status update %d\n", i)  
            c.SSEvent("message", msg)  
            time.Sleep(2 * time.Second)  
        }  
    }  
}
```

01 Endpoint SSE di Gin

Buat endpoint yang mengirim event SSE setiap 1 detik untuk total 10 kali pengiriman update status proses.

03 Hentikan saat client putus

Tambahkan deteksi koneksi client terputus agar loop server berhenti otomatis dan tidak terus berjalan di server.

02 Payload langkah dan status

Setiap data harus memuat nomor langkah serta string status agar progres proses mudah dipantau oleh client.

WebSocket Intermediate

```
func main() {  
    // ...  
    var clients = make(map[*websocket.Conn]bool)  
    var mu sync.Mutex  
  
    func handleWebSocket(c *gin.Context) {  
        conn, err := websocket.Upgrader{CheckOrigin: func(*http.Request) bool {  
            return true  
        }}.Upgrade(c.Writer, c.Request, nil)  
        if err != nil {  
            c.String(http.StatusInternalServerError, "Upgrade failed")  
            return  
        }  
        mu.Lock()  
        clients[conn] = true  
        mu.Unlock()  
        defer func() {  
            mu.Lock()  
            delete(clients, conn)  
            mu.Unlock()  
            conn.Close()  
        }()  
    }  
}
```

01 Endpoint chat WebSocket

Buat endpoint WebSocket sederhana dengan Gin untuk menerima pesan client dan mengirim balasan.

03 Broadcast multi-client

Kembangkan server agar menangani banyak koneksi aktif dan menyiarkan pesan ke semua client terhubung.

02 Tambahkan timestamp

Format setiap pesan keluar dengan tambahan timestamp agar waktu kirim terlihat jelas.

CLOUD SERVICES

LIVE DATA STREAMS



DATABASES

Ringkasan



"SSE dan WebSocket sama-sama mendukung komunikasi real-time, tetapi SSE cocok untuk aliran satu arah sederhana, sedangkan WebSocket untuk interaksi dua arah."

MESSAGE QUEUES

REAL-TIME PROCESSING

Ringkasan Teknologi



API CENDREDC

